



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 5

по курсу «Разработка параллельных и распределенных программ»

Студент: Ивенкова О. В.

Группа: ИУ9-52Б

Преподаватель: Царёв А. С.

Москва, 2025

Содержание

1	Постановка задачи	3
2	Практическая реализация	4
3	Характеристики устройства	14

1 Постановка задачи

Лабораторная работа №5. Синхронизация потоков

Для выполнения лабораторной работы необходимо решить две следующие задачи (можно в одной программе, можно в двух отдельных).

1) Использование барьерной синхронизации - задача "эволюция". Дан двумерный массив клеток, каждая из которых либо содержит организм (1), либо пуста (0), изначально он заполняется случайными значениями. Каждая клетка проверяет состояние своих соседей (их 8) и изменяет своё по правилам:

- Живая клетка, вокруг которой < 2 живых клеток, умирает от одиночества.
- Живая клетка, вокруг которой есть 2 или 3 живых клеток, выживает.
- Живая клетка, вокруг которой > 3 живых клеток, умирает от перенаселения.
- Пустая клетка, рядом с которой равно 3 живых соседа, оживает.

Реализовать заданное количество шагов моделирования при помощи n потоков. Каждый поток должен вычислить значения в заданной ему полосе матрицы. На каждом шаге результат моделирования необходимо записывать в новую матрицу. По окончании очередного шага необходимо скопировать содержимое новой матрицы в исходную. Шаги между потоками синхронизировать с помощью барьера (ни один из потоков не должен начинать следующий шаг, пока все не закончили текущий). Учесть следующие моменты:

- у клеток, находящихся на первой строке, первом столбце, последней строке и последнем столбце, соседями являются клетки с противоположной стороны матрицы
- каждый поток видит только свою часть матрицы, поэтому если ему необходим элемент, которого в его части матрицы нет, он должен каким-либо образом составлять запрос на то, чтобы тот поток, в котором этот элемент есть, его ему предоставил.

Замерить среднее время выполнения одного шага алгоритма и сравнить со средним временем выполнения одного шага без использования потоков. Результат внести в отчёт.

2) Использование синхронизаций чтения и записи. Написать программу, в которой каждый из n потоков генерирует заданное количество псевдослучайных чисел от 0 до 1000, каждое из которых он должен помещать в конец однонаправленного связного списка, но только если такого числа там ещё нет. Синхронизацию доступа к списку необходимо выполнять при помощи двух отдельных блокировок: чтения и записи. Проверку числа на принадлежность списку необходимо выполнить в блокировке на чтение. Далее, если число не обнаружено, выполнить блокировку на запись и перед добавлением числа вновь проверить его на принадлежность списку. По окончании работы потоков ещё раз основным потоком проверить отсутствие повторяющихся чисел в списке. В отчёт внести только код программы.

2 Практическая реализация

Исходный код программы представлен в листингах 1– 2.

Листинг 1: Код первой программы

```
1 #include <algorithm>
2 #include <atomic>
3 #include <chrono>
4 #include <condition_variable>
5 #include <cstdint>
6 #include <iostream>
7 #include <mutex>
8 #include <random>
9 #include <thread>
10 #include <vector>
11
12 using Clock = std::chrono::high_resolution_clock;
13 using std::chrono::duration_cast;
14 using std::chrono::microseconds;
15 using namespace std::chrono_literals;
16
17 class Barrier {
18 public:
```

```

19     explicit Barrier(unsigned count) : thread_count(count), counter(0),
    waiting(0) {}
20     void wait() {
21         std::unique_lock<std::mutex> lk(mtx);
22         unsigned gen = counter;
23         if (++waiting == thread_count) {
24             waiting = 0;
25             ++counter;
26             cv.notify_all();
27         } else {
28             cv.wait(lk, [&] { return counter != gen; });
29         }
30     }
31
32 private:
33     std::mutex mtx;
34     std::condition_variable cv;
35     const unsigned thread_count;
36     unsigned counter;
37     unsigned waiting;
38 };
39
40 using Grid = std::vector<std::vector<uint8_t>>>;
41
42 inline int count_neighbors_global(const Grid &grid, int rows, int cols,
    int r, int c) {
43     int cnt = 0;
44     for (int dr = -1; dr <= 1; ++dr) {
45         int rr = (r + dr + rows) % rows;
46         for (int dc = -1; dc <= 1; ++dc) {
47             if (dr == 0 && dc == 0) continue;
48             int cc = (c + dc + cols) % cols;
49             cnt += grid[rr][cc];
50         }
51     }
52     return cnt;
53 }
54
55 inline int count_neighbors_local(const std::vector<std::vector<uint8_t>>>
    &local,
56     const std::vector<uint8_t> &top_halo,
57     const std::vector<uint8_t> &bottom_halo,
58     int rows_local, int cols, int local_r, int c) {
59     int cnt = 0;
60     for (int dr = -1; dr <= 1; ++dr) {
61         int rr = local_r + dr;

```

```

62         for (int dc = -1; dc <= 1; ++dc) {
63             if (dr == 0 && dc == 0) continue;
64             int cc = (c + dc + cols) % cols;
65             if (rr < 0) {
66                 cnt += top_halo[cc];
67             } else if (rr >= rows_local) {
68                 cnt += bottom_halo[cc];
69             } else {
70                 cnt += local[rr][cc];
71             }
72         }
73     }
74     return cnt;
75 }
76
77 void random_init(Grid &grid, int rows, int cols, double alive_prob =
    0.5) {
78     std::mt19937_64 rng(std::random_device{}());
79     std::uniform_real_distribution<double> dist(0.0, 1.0);
80     for (int r = 0; r < rows; ++r) {
81         for (int c = 0; c < cols; ++c) {
82             grid[r][c] = (dist(rng) < alive_prob) ? 1 : 0;
83         }
84     }
85 }
86
87 void step_single_thread(const Grid &current, Grid &next, int rows, int
    cols) {
88     for (int r = 0; r < rows; ++r) {
89         for (int c = 0; c < cols; ++c) {
90             int n = count_neighbors_global(current, rows, cols, r, c);
91             if (current[r][c]) {
92                 next[r][c] = (n == 2 || n == 3) ? 1 : 0;
93             } else {
94                 next[r][c] = (n == 3) ? 1 : 0;
95             }
96         }
97     }
98 }
99
100 std::vector<std::pair<int, int>> partition_stripes(int rows, int nthreads
    ) {
101     std::vector<std::pair<int, int>> parts;
102     parts.reserve(nthreads);
103     int base = rows / nthreads;
104     int rem = rows % nthreads;

```

```

105     int cur = 0;
106     for (int i = 0; i < nthreads; ++i) {
107         int size = base + (i < rem ? 1 : 0);
108         int s = cur;
109         int e = cur + size - 1;
110         if (size == 0) { s = 0; e = -1; }
111         parts.emplace_back(s, e);
112         cur += size;
113     }
114     return parts;
115 }
116
117 double run_multithread(Grid grid_current, Grid grid_next, int rows, int
    cols, int steps, int nthreads) {
118     if (nthreads <= 0) nthreads = 1;
119     auto parts = partition_stripes(rows, nthreads);
120
121     std::vector<std::vector<uint8_t>> top_rows(nthreads, std::vector<
    uint8_t>(cols));
122     std::vector<std::vector<uint8_t>> bottom_rows(nthreads, std::vector<
    uint8_t>(cols));
123     std::vector<std::mutex> top_mutex(nthreads), bottom_mutex(nthreads);
124
125     Barrier barrier_send(nthreads);
126     Barrier barrier_compute(nthreads);
127     Barrier barrier_swap(nthreads);
128
129     std::atomic<bool> stop_flag{false};
130
131     auto worker = [&](int tid) {
132         auto [srow, erow] = parts[tid];
133         if (srow > erow) {
134             for (int step = 0; step < steps; ++step) {
135                 barrier_send.wait();
136                 barrier_compute.wait();
137                 barrier_swap.wait();
138             }
139             return;
140         }
141         int rows_local = erow - srow + 1;
142         std::vector<std::vector<uint8_t>> local(rows_local, std::vector<
    uint8_t>(cols));
143         std::vector<uint8_t> top_halo(cols), bottom_halo(cols);
144
145         for (int step = 0; step < steps; ++step) {
146             {

```

```

147         std::lock_guard<std::mutex> lk(top_mutex[tid]);
148         for (int c = 0; c < cols; ++c) top_rows[tid][c] =
grid_current[srow][c];
149     }
150     {
151         std::lock_guard<std::mutex> lk(bottom_mutex[tid]);
152         for (int c = 0; c < cols; ++c) bottom_rows[tid][c] =
grid_current[erow][c];
153     }
154
155     barrier_send.wait();
156
157     int prev_tid = tid - 1;
158     while (prev_tid >= 0 && parts[prev_tid].first > parts[tid].
first - 1) --prev_tid;
159
160     auto owner_of_row = [&](int row)->int {
161         if (row < 0) row += rows;
162         if (row >= rows) row -= rows;
163         for (int i = 0; i < nthreads; ++i) {
164             if (parts[i].first <= row && row <= parts[i].second)
return i;
165         }
166         return -1;
167     };
168     int owner_top = owner_of_row(srow - 1);
169     int owner_bottom = owner_of_row(erow + 1);
170     if (owner_top == -1 || owner_bottom == -1) {
171         std::cerr << "Error: the owner of the row was not found
when calculating the halo.\n";
172         return;
173     }
174     {
175         std::lock_guard<std::mutex> lk(bottom_mutex[owner_top]);
176         for (int c = 0; c < cols; ++c) top_halo[c] = bottom_rows
[owner_top][c];
177     }
178     {
179         std::lock_guard<std::mutex> lk(top_mutex[owner_bottom]);
180         for (int c = 0; c < cols; ++c) bottom_halo[c] = top_rows
[owner_bottom][c];
181     }
182
183     for (int i = 0; i < rows_local; ++i)
184         for (int c = 0; c < cols; ++c)
185             local[i][c] = grid_current[srow + i][c];

```



```

186
187         for (int local_r = 0; local_r < rows_local; ++local_r) {
188             int global_r = srow + local_r;
189             for (int c = 0; c < cols; ++c) {
190                 int n = count_neighbors_local(local, top_halo,
bottom_halo, rows_local, cols, local_r, c);
191                 uint8_t cur = local[local_r][c];
192                 uint8_t out = 0;
193                 if (cur) {
194                     out = (n == 2 || n == 3) ? 1 : 0;
195                 } else {
196                     out = (n == 3) ? 1 : 0;
197                 }
198                 grid_next[global_r][c] = out;
199             }
200         }
201
202         barrier_compute.wait();
203
204         if (tid == 0) {
205             for (int r = 0; r < rows; ++r)
206                 for (int c = 0; c < cols; ++c)
207                     grid_current[r][c] = grid_next[r][c];
208         }
209
210         barrier_swap.wait();
211     }
212 };
213
214 std::vector<std::thread> threads;
215 threads.reserve(nthreads);
216
217 auto t0 = Clock::now();
218
219 for (int i = 0; i < nthreads; ++i) threads.emplace_back(worker, i);
220
221 for (auto &th : threads) th.join();
222
223 auto t1 = Clock::now();
224 auto total_us = duration_cast<microseconds>(t1 - t0).count();
225 double avg_per_step = double(total_us) / steps;
226
227
228 return avg_per_step;
229 }
230

```

```

231 int main() {
232     int rows = 400;
233     int cols = 400;
234     int steps = 200;
235     int nthreads = 8;
236
237     std::cout << "Size: " << rows << "x" << cols << ", steps: " << steps
238     << ", threads: " << nthreads << "\n";
239
240     Grid grid_current(rows, std::vector<uint8_t>(cols));
241     Grid grid_next(rows, std::vector<uint8_t>(cols));
242     random_init(grid_current, rows, cols, 0.3);
243
244     double avg_mt_us = run_multithread(grid_current, grid_next, rows,
245     cols, steps, nthreads);
246     std::cout << "Multithreaded: average time per step = " << avg_mt_us
247     * pow(10, 6) << " s\n";
248
249     Grid grid_single_current = grid_current;
250     Grid grid_single_next(rows, std::vector<uint8_t>(cols));
251
252     auto t0 = Clock::now();
253     for (int step = 0; step < steps; ++step) {
254         step_single_thread(grid_single_current, grid_single_next, rows,
255         cols);
256         for (int r = 0; r < rows; ++r)
257             for (int c = 0; c < cols; ++c)
258                 grid_single_current[r][c] = grid_single_next[r][c];
259     }
260     auto t1 = Clock::now();
261     auto total_us = duration_cast<microseconds>(t1 - t0).count();
262     double avg_st_us = double(total_us) / steps;
263     std::cout << "Single-threaded: average time per step = " <<
264     avg_st_us * pow(10, 6) << " s\n";
265
266     return 0;
267 }

```

Вывод программы:

Size: 10000x10000, steps: 10, threads: 8

Multithreaded: average time per step = 9.15705 s

Single-threaded: average time per step = 30.82114 s

Листинг 2: Код второй программы

```
1 #include <iostream>
2 #include <thread>
3 #include <vector>
4 #include <mutex>
5 #include <shared_mutex>
6 #include <random>
7 #include <unordered_set>
8
9 struct Node {
10     int value;
11     Node* next;
12     Node(int v) : value(v), next(nullptr) {}
13 };
14
15 Node* head = nullptr;
16
17 std::mutex read_lock;
18 std::mutex write_lock;
19
20 bool contains_read(int x) {
21     std::lock_guard<std::mutex> lock(read_lock);
22
23     Node* cur = head;
24     while (cur) {
25         if (cur->value == x) {
26             return true;
27         }
28         cur = cur->next;
29     }
30
31     return false;
32 }
33
34 void insert_if_not_exists(int x) {
35     std::lock_guard<std::mutex> w_lock(write_lock);
36
37     {
38         Node* cur = head;
39         while (cur) {
40             if (cur->value == x) return;
41             cur = cur->next;
42         }
43     }
44     Node* n = new Node(x);
45     if (!head) {
```

```

46         head = n;
47         return;
48     }
49     Node* cur = head;
50     while (cur->next) cur = cur->next;
51     cur->next = n;
52 }
53
54 void worker(int count_per_thread) {
55     std::mt19937 rng(std::random_device{}());
56     std::uniform_int_distribution<int> dist(0, 1000);
57
58     for (int i = 0; i < count_per_thread; i++) {
59         int x = dist(rng);
60
61         if (!contains_read(x)) {
62             insert_if_not_exists(x);
63         }
64     }
65 }
66
67 bool check_duplicates() {
68     std::lock_guard<std::mutex> lock(read_lock);
69     std::unordered_set<int> seen;
70     Node* cur = head;
71     while (cur) {
72         if (!seen.insert(cur->value).second)
73             return false;
74         cur = cur->next;
75     }
76     return true;
77 }
78
79 void cleanup() {
80     std::lock_guard<std::mutex> w_lock(write_lock);
81     std::lock_guard<std::mutex> r_lock(read_lock);
82
83     Node* cur = head;
84     while (cur) {
85         Node* temp = cur;
86         cur = cur->next;
87         delete temp;
88     }
89     head = nullptr;
90 }
91

```

```

92 int main() {
93     int nthreads = 8;
94     int per_thread = 10000;
95
96     std::vector<std::thread> threads;
97     threads.reserve(nthreads);
98
99     for (int i = 0; i < nthreads; i++)
100         threads.emplace_back(worker, per_thread);
101
102     for (auto& t : threads) t.join();
103
104     //print_list();
105
106     if (check_duplicates())
107         std::cout << "There are no duplicate numbers\n";
108     else
109         std::cout << "Duplicates found\n";
110
111     cleanup();
112     return 0;
113 }

```

3 Характеристики устройства

Имя устройства Acer Extensa 215-54

Оперативная память 16,0 ГБ

Процессор устройства Intel Core i5-1135G7 (2.40 ГГц, 4 ядра, 15 Вт)